

```

1 #include <stdio.h>
2
3 typedef struct {
4     char name[10];
5     char type[20];
6 } Instruction;
7
8 void printInstruction(Instruction instruction) {
9
10     printf("%-10s %-20s\n",
11           instruction.name,
12           instruction.type);
13 }
14
15 int main() {
16
17     Instruction instructions[] = {
18
19         {"ADD R1,R2,R3", "Normal"},
20         {"SUB R4,R1,R5", "Data Hazard"},
21         {"MUL R6,R4,R7", "Data Hazard"},
22         {"BEQ R6,R0", "Control Hazard"},
23         {"LOAD R8,100", "Structural Hazard"},
24         {"ADD R9,R8,R10", "Data Hazard"}
25
26     };
27
28     int count = sizeof(instructions) /
29               sizeof(instructions[0]);
30

```

```

BEQ R6,R0    CONTROL HAZARD
LOAD R8,100  Structural Hazard Structural Hazard
ADD R9,R8,R10 Data Hazard Data Hazard

```

HAZARD ANALYSIS

1. DATA HAZARD

Instruction 1: ADD R1,R2,R3
Instruction 2: SUB R4,R1,R5

The SUB instruction requires R1,
but R1 is produced by the previous ADD.

Input

Enter your input here...

```

1 #include <stdio.h>
2
3 #define INSTRUCTIONS 8
4 #define STAGES 5
5
6 int main() {
7
8     int pipeline[STAGES][INSTRUCTIONS] = {0};
9
10    int i, j;
11
12    printf("=====\n");
13    printf("5-STAGE INSTRUCTION PIPELINE SIMULATOR\n");
14    printf("=====\n");
15
16    printf("\nInstructions = %d", INSTRUCTIONS);
17    printf("\nPipeline Stages = 5");
18    printf("\n");
19
20    /*
21     Pipeline stages:
22     0 = IF
23     1 = ID
24     2 = EX
25     3 = MEM
26     4 = WB
27    */
28
29    for (i = 0; i < INSTRUCTIONS; i++) {
30

```

5-STAGE INSTRUCTION PIPELINE SIMULATOR

Instructions = 8
Pipeline Stages = 5

Pipeline Execution:

Cycle	IF	ID	EX	MEM	WB
1	I1	-	-	-	-
2	I2	I1	-	-	-
3	I3	I2	I1	-	-

Input

Enter your input here...

Online C Compiler

Share

Run

```
1 #include <stdio.h>
2
3 typedef struct {
4     int R0;
5     int R1;
6     int R2;
7     int R3;
8     int R4;
9 } Registers;
10
11 void displayRegisters(Registers r) {
12     printf("\n-----\n");
13     printf("Register Status\n");
14     printf("-----\n");
15     printf("R0 = %d\n", r.R0);
16     printf("R1 = %d\n", r.R1);
17     printf("R2 = %d\n", r.R2);
18     printf("R3 = %d\n", r.R3);
19     printf("R4 = %d\n", r.R4);
20 }
21
22 int main() {
23     Registers r = {0, 0, 0, 0, 0};
24
25     int choice;
26
27     printf("===== \n");
28     printf(" REGISTER TRANSFER AND ALU SIMULATOR\n");
29     printf("===== \n");
30 }
```

Output

=====

REGISTER TRANSFER AND ALU SIMULATOR

=====

Enter value for R1: Enter value for R2:

Register Status

R0 = 0

R1 = 20

R2 = 10

R3 = 0

R4 = 0

=====

Input

20 10

```
1 #include <stdio.h>
2
3 typedef struct {
4     int R1;
5     int R2;
6     int R3;
7     int R4;
8 } Registers;
9
10 void singleBusSimulation() {
11     Registers r = {10, 20, 0, 0};
12
13     int cycles = 0;
14
15     printf("\n===== \n");
16     printf(" SINGLE-BUS ORGANIZATION\n");
17     printf("===== \n");
18
19     printf("\nInitial values:");
20     printf("\nR1 = %d", r.R1);
21     printf("\nR2 = %d", r.R2);
22
23     printf("\n\nTransfer R1 -> R3");
24     r.R3 = r.R1;
25     cycles++;
26
27     printf("\n\nTransfer R2 -> R4");
28     r.R4 = r.R2;
29     cycles++;
30 }
```

SINGLE-BUS ORGANIZATION

=====

Initial values:

R1 = 10

R2 = 20

Transfer R1 -> R3

Transfer R2 -> R4

Memory -> R1

R2 -> I/O

Final Register Values:

R1 = 100

R2 = 20

R3 = 10

R4 = 20

Input

Enter your input here...

<pre> 1 #include <stdio.h> 2 3 #define INSTRUCTIONS 12 4 5 int main() { 6 7 int instructions = INSTRUCTIONS; 8 9 int scalarWidth = 1; 10 int superscalarWidth = 2; 11 int smtThreads = 2; 12 13 int scalarCycles; 14 int superscalarCycles; 15 int smtCycles; 16 17 double scalarThroughput; 18 double superscalarThroughput; 19 double smtThroughput; 20 21 printf("=====\n"); 22 printf(" SUPERSCALAR AND SMT PROCESSOR SIMULATOR\n"); 23 printf("=====\n"); 24 25 printf("\nNumber of instructions = %d", instructions); 26 printf("\nSuperscalar width = %d", superscalarWidth); 27 printf("\nSMT threads = %d", smtThreads); 28 29 /* 30 Scalar processor </pre>	<pre> ===== SUPERSCALAR AND SMT PROCESSOR SIMULATOR ===== Number of instructions = 12 Superscalar width = 2 SMT threads = 2 ===== SCALAR EXECUTION ===== Instructions per cycle = 1 Total cycles = 12 Throughput = 1.00 instructions/cycle </pre>
	Input
	Enter your input here...